When Does INT8 Compression Preserve Federated Intrusion Detection on Microcontrollers?

Input Ranges, Operating Points, and Client-Data Constraints

Anonymous Author(s)

Abstract

Federated intrusion detectors are trained on server-class hardware but must run on microcontrollers with tens of kilobytes of memory. We ask when compressing such a detector to INT8 preserves its usefulness at a fixed false-alarm rate, why it fails when it does, and which remedies work when only client-held data are available. On CIC-IDS2017 and TON_IoT, with a data pipeline audited for client ownership and test isolation, thresholds chosen for a 1% false-alarm budget on client validation data, and three independent federated runs, the best 57–65 KB INT8 models detect 98.5% and 99.6% of test attacks at an achieved test false-alarm rate of 1.16% and 0.92%. A compact model trained federatively from scratch matches the complete recovery recipe (distillation, clipped inputs, QAT) on CIC-IDS2017 and on unseen hosts and trails it by 0.5–1.6 points on TON_IoT. Failures have one dominant cause, the input quantization range: ten extreme records in a 2,000-record calibration draw drive recall to 0%, and quantization-aware training (QAT) fails for the same reason because it learns that range from unclipped inputs. Bounding the input range removes both failures at no device cost, although clipping saturates features of 8–28% of test records. On hosts never seen in training, the compact INT8 models lose 1.6–4.4 points of recall relative to the float teacher. The residual loss lies in coarse activations near the operating point and is recovered on the host by 16-bit activations. Finally, the exact exported file and runtime decide what the device does: on TFLM 1.0.0 the per-tensor INT8 models reproduce the host decisions on an ESP32-S3 and an ESP32 (5.6–6.5 and 13.2–15.4 ms per flow), while the converter's default per-channel export silently disagrees on half of its decisions; the current Espressif runtime restores agreement for the per-channel model we tested, and the 16-bit model runs correctly on neither runtime.

Keywords: federated learning, TinyML, intrusion detection, quantization, microcontrollers

1 Introduction

Intrusion detection on IoT devices is increasingly learned from traffic that clients do not want to pool [1]. Federated learning (FL) trains a shared model from client updates [23, 29], and TinyML runs integer models on microcontrollers [2, 8]. Putting the two together requires a step that is easy to state and easy to get wrong: the federated model has to be compressed, usually to INT8 and a fraction of its size, without pooled data, and the compressed model has to keep detecting attacks at the false-alarm rate an operator can tolerate.

Published federated and TinyML intrusion detectors report accuracy or F1 at a fixed threshold, on splits whose preprocessing often sees the test data, and rarely check the device. We found all three choices in our own earlier pipeline: oversampling before the client partition, thresholds tuned on synthetic records, and a test split that guided method development (§3). After rebuilding the pipeline we ask three questions:

RQ1 When does compression preserve useful detection, under memory budgets of 80 and 128 KB and false-alarm (FAR) budgets of 1% and 5% chosen on client validation data?

RQ2 Why does it fail when it does?

RQ3 Which remedies work under memory and client-data constraints, and what do they cost on the device?

On CIC-IDS2017 and TON_IoT with three federated runs per configuration, the answers form three connected findings:

- **A compact model trained from scratch is often enough (RQ1, RQ3).** At 57–65 KB, a model trained federatively at the target size (S80) matches the complete teacher-based recovery recipe R (pruning, distilled fine-tuning on clipped inputs, QAT on clipped inputs) on CIC-IDS2017 and on unseen hosts; R is better on the random TON_IoT split by 0.5–1.6 points in every run, at a 0.1–0.2-point higher achieved test FAR. Both have the same architecture, size and latency; they differ in training cost (§4, Table 3).
- **The input quantization range causes and prevents the failures (RQ2, RQ3).** A controlled 2×2 isolates the input range, not hidden ranges, as the cause of collapses; the same mechanism explains why QAT fails; bounding the range (clipped or extreme-free calibration, QAT on clipped inputs) removes the failures at no device cost, with an accuracy trade-off for models that use extreme inputs. The remaining loss comes from tie groups of coarse INT8 outputs and is recovered on the host, not on the device, by 16-bit activations (§5, §6).
- **Detection must be verified for the exact exported model and runtime (§7).** On TFLM 1.0.0 the per-tensor INT8 models reproduce the host decisions on two ESP32 chips, while per-channel exports of the

same models silently do not. On the classic ESP32 the current Espressif runtime ran the tested per-channel model correctly and gave bit-identical outputs for its per-tensor counterpart; the 16-bit model ran correctly on neither runtime.

We also contribute the audited protocol itself: client-owned partitions before any fitted step, real per-client validation slices, FAR-budget thresholds applied once to a test split that was excluded from training, threshold selection and calibration of every reported model (its records had been in the training pool of our earlier, exploratory pipeline; §3.1), and an unseen-host split. Per-run results, paired differences, device disagreement classes, resource breakdowns and stream-completeness records are in the supplementary material (S1–S8).

2 Background and Related Work

Datasets. CIC-IDS2017 [39] is the most used flow-based IDS benchmark; Engelen et al. [11] document labelling and flow-construction errors in it. TON_IoT [31] and Bot-IoT [24] target IoT traffic.

Federated and lightweight intrusion detection. DÏoT [35] trains per-device-type detectors with FL; Belarbi et al. [4] use one TON_IoT client per IP address; FLAME [36] defends FL against backdoors. Li et al. [25] combine FL and TinyML for intrusion detection on N-BaIoT with feature reduction and quantization. Kitsune [30], HorusEye [10] and NetBeacon [42] run detectors on gateways or switches; we target the endpoint microcontroller. We use FedAvgM [18] in Flower [5].

TinyML and compression. TFLM [8] runs integer models on microcontrollers; MCUNet [26], on-device training [27], SeeDot [14] and MinUn [21] co-design models and runtimes. HeteroFL [9] and FedTiny [19] produce small models during federated training; we compress a trained federated model or train a compact one from scratch. Pruning [16], distillation [17, 37] and quantization [13, 20, 32] are standard; QAT can oscillate [33], and pruned weights re-densify under standard QAT [40]. In FL, quantization mostly compresses updates [38]; Gupta et al. train models robust to later quantization [15].

Calibration ranges. Percentile and KL calibration [41] and the MSE-optimal clip of ACIQ [3] clip activation ranges instead of using extremes; PACT [7] and LSQ [12] learn the clip or step during QAT, and DFQ [34] calibrates without data. These methods target hidden activations; we find that for tabular flow features the *input* range decides the outcome. Deployment faults that yield wrong models without errors are common [6, 22]; §7 adds one for microcontroller runtimes.

3 Experimental Design

3.1 Data, Audit and Splits

We use the eight CIC-IDS2017 CSV files (78 features, binary label) and the TON_IoT network train/test file (37 numeric and label-encoded fields plus six length / digit / symbol counts from the DNS query and URI). Table 1 summarizes the audited pipeline. An audit of our earlier pipeline (Appendix A) found that SMOTE ran on the pooled training set before the client partition (37.5% / 35.1% synthetic rows, with synthetic descendants of one client's records on others), that the validation slice used to choose thresholds was part of a client's training data and 74.6% (CIC-IDS2017 attack rows) / 71.0% (TON_IoT benign rows) synthetic, that 4.7% of synthetic TON_IoT rows carried values impossible for their label-encoded fields, and that the test split had guided method choices. All results in this paper come from the rebuilt pipeline:

1. exact duplicates are removed and feature vectors that occur with both labels are dropped; on CIC-IDS2017, 2,926 records with impossible negative values (overflowed header lengths down to $-3.2 \cdot 10^{10}$, negative durations and inter-arrival times) are dropped as data errors, while the defined -1 sentinel of `Init_Win_bytes` is kept;
2. the test split (20%) is drawn first, only from records that were not in the earlier test split;
3. records are assigned to four clients (label-balanced) before any fitted step; each client holds out a real, stratified 10% validation slice and undersamples its own training slice to at most 4:1;
4. the scaler uses the mean and variance of the union of client training slices, which a federation can aggregate from per-client sums; validation and test are never used.

Test isolation and freeze history. All revised test records come from the training pool of our earlier, exploratory pipeline: models of that pipeline saw them in training and guided our hypotheses, but none of the records was in the earlier test split (Supplement S7a). They are excluded from training, threshold selection and calibration of every reported model, and the record ids and hashes of every partition are stored per run and checked before evaluation. The pipeline, arms, budgets, threshold rule and calibration candidates were committed before any revised result, and the rules that make the test split exclude the earlier test records before any revised test result. Added after test results, and marked as post hoc: the QC/SQC arms, the clipped-FP32 and 16-bit rows of E4, the unseen-host split, the per-tensor re-exports and the runtime comparison (time line in Supplement S7b).

For a second test condition (E6) we split TON_IoT by source host: all records of 15 of 51 source IPs form the test split, so no test host is seen in training. CIC-IDS2017 carries no host field in its CSVs.

Table 1. Audited data pipeline (per dataset). Val. = per-client validation slice (benign count in brackets).

	CIC-IDS2017	TON_IoT
raw records	2,830,743	211,043
data errors dropped	2,926	0
exact duplicates removed	331,190	105,867 (50%)
conflicting-label vectors (records)	719 (7,144)	8 (314)
records kept	2,495,189	105,160
test split (attacks / benign)	84,990 / 414,048	16,212 / 4,820
training slice per client (attacks)	382,455 (76,491)	18,929 (14,592)
val. per client (benign)	49,904 (41,405)	2,103 (482)
features	78	43

3.2 Federated Training

All models are MLPs with ReLU, post-ReLU BatchNorm (folded exactly into the next layer before compression, Appendix B) and dropout, trained with Flower and FedAvgM (server momentum 0.5, server learning rate 0.1) for 60 rounds of 3 local epochs, Adam with cosine decay from 10^{-3} to 10^{-4}, class weights and focal loss [28] ($\alpha = 0.35$ on CIC-IDS2017, 0.5 on TON_IoT). The teacher F is 512-256-128. Compact models trained from scratch with the same recipe are S80 (256-128-64, 57 KB / 65 KB as INT8) and S128 (358-179-89, 100 KB / 112 KB). Every configuration is trained with seeds 1, 2 and 3 (E5); a run takes 3–4 min on TON_IoT and 26–41 min on CIC-IDS2017 on one PC.

3.3 Compared Arms

For each teacher seed and memory budget (80 KB: structured pruning keeps 50% of the neurons; 128 KB: 70%) we compare:

- **F / F8** the float teacher, and the teacher with strong PTQ (189 / 207 KB, over budget; a reference);
- **P** prune → fine-tune 3 FedAvg rounds on the clients' data → strong PTQ;
- **Q** the same pruned and fine-tuned model → QAT for 2 rounds → INT8;
- **QC** as Q, but QAT on inputs clipped to ±5 standard deviations;
- **R** prune → fine-tune distilled from F (temperature 2, $\alpha = 0.5$) on clipped inputs → QAT on clipped inputs → INT8;
- **S / SQ / SQC** the from-scratch compact model with strong PTQ / QAT / QAT on clipped inputs.

Strong PTQ builds seven calibrations from 2,000 records of client 0 (TFLite min/max with and without ±5 SD input clipping, per-tensor 99.9 / 99.99th percentile, MSE [3], KL [41], clipped 99.99th percentile) and keeps the one with the best mean validation recall at the two FAR budgets; reported F8, P and S80 models are restricted to the per-tensor candidates, the form that TFLM 1.0.0 executes correctly (§7). Fine-tuning uses 10k records per client, either FedAvg over all four clients or one client alone (the client-data constraint).

Table 2. Attack recall (%) at a 1% FAR budget chosen on client validation data, mean ± SD over three federated runs; test FAR in brackets. 80 KB budget; fine-tuning FedAvg over four clients (10k records each). 1-cl.: fine-tuning on one client's 10k records (test FAR 0.91–1.31%). F8, P and S80 use strong PTQ restricted to per-tensor weight scales, the form TFLM 1.0.0 executes correctly; allowing per-channel exports changes their mean recall by at most 0.2 points (†: mean includes per-channel exports). Per-run values, both budgets and the exported files with SHA-256: Supplement S1.

arm	KB	TON_IoT	CIC-IDS2017	1-cl. TON_IoT/ CIC-IDS2017
F (FP32)	–	96.68±1.38 [0.84]	96.89±2.05 [0.98]	
F8	188/206	98.35±0.06 [1.19]	99.33±0.54 [0.90]	
P	57/65	96.57±1.89 [0.98]	99.12±0.37 [0.92]	96.92 / 99.06
Q	57/65	55.4±48.8 [0.61]	81.9±9.2 [0.80]	59.6 / 88.5
QC	57/65	97.86±0.44 [1.01]	99.57±0.10 [0.94]	
R	57/65	**98.51**±0.11 [1.16]	**99.60**±0.15 [0.92]	97.80 / 99.52
S80	57/65	97.57±0.52 [1.01]	99.57±0.24 [0.97]	
SQ	57/65	59.0±51.1 [0.55]	84.8±5.4 [0.82]	
SQC	57/65	97.71±1.09 [1.01]	99.59±0.07 [0.97]	
128 KB budget: P 97.78 / 99.19, QC 98.24 / 99.56, R 98.72 / 99.64, S128† 98.58 / 99.37				

3.4 Operating Points and Metrics

For every model and stage, the threshold is the smallest one whose FAR on the pooled client validation slices is at most the budget (1% or 5%). It is frozen and applied once to the test split; we report attack recall and the *achieved* test FAR, mean ± SD over the three teacher seeds. The budget constrains validation FAR only; test FAR can exceed it (on TON_IoT it reaches 1.27% at the 1% budget) and, on unseen hosts, differs from it by several points (§4). Decision transitions (E2) compare each test attack's decision at consecutive stages (teacher, compact FP32, QAT float, INT8), each at its own frozen threshold. Thresholds are computed in float32: with NumPy 1.x a float64 threshold is cast back to float32 in the comparison, which silently re-admits a whole tie group of INT8 outputs and let validation FAR reach 44% at a 5% budget in our first run.

4 RQ1: When Compression Preserves Detection

Table 2 gives attack recall at a validation-chosen 1% FAR budget with the achieved test FAR. At the 5% budget every arm except the QAT arms Q and SQ lies within 0.2 points of the teacher (99.7–99.9% on TON_IoT, 99.8–99.97% on CIC-IDS2017) at achieved test FARs of 3.5–4.4% (TON_IoT) and 3.2–5.1% (CIC-IDS2017).

Compression preserves detection when the input range is bounded. Every arm whose input range is bounded (P with strong PTQ, QC, R, S80, SQC) lies within one SD of the teacher's recall at the 1% budget or above it; the means of R, QC, S80 and SQC exceed the teacher's on both datasets,

at achieved test FARs of 0.92–1.16% against the teacher's 0.84–0.98%. The 128 KB budget adds at most 1.2 points over 80 KB. The two QAT arms that learn their input range from unclipped data (Q, SQ) fail; §5 explains why.

The float teacher's 1% operating point is the least stable. F has the largest SD (1.4 and 2.1 points). On CIC-IDS2017 about 2,265 PortScan flows per run sit just below its 1% threshold, and every compact or INT8 model recovers them (§5.1). This is an operating-point sensitivity of the teacher, not a benefit of compression (§5.3).

Choosing a deployable model. Table 3 consolidates the principal deployable models. Pruning half of the teacher's neurons gives P, QC and R the same 256-128-64 architecture as the from-scratch model S80, so all five have the same size and device latency; they differ in accuracy, achieved FAR and training cost. The compact model trained from scratch is sufficient on CIC-IDS2017 (R − S80 between −0.27 and +0.41 points per run) and on unseen hosts (−1.24 to +1.62). The complete recovery recipe R is better only on the random TON_IoT split, by 0.48–1.62 points in every run, and there its achieved test FAR is also 0.10–0.19 points higher. S80 needs its own 60-round federated training (2.7 min per run on TON_IoT, 25.9 min on CIC-IDS2017, 64–79% of the teacher's) but no teacher; R needs the trained teacher plus three distilled fine-tuning rounds, which pass every fine-tuning record through the teacher, and two QAT rounds. When a teacher already exists, R adds five rounds; when the deployment size is known before training, S80 avoids training the teacher at all. The plain pipeline P is below S80 in every run on CIC-IDS2017 and on unseen hosts. Run-paired differences for all pairs: Supplement S2.

One client's data sufficed in our partition. In our label-balanced four-client partition, fine-tuning P and R on one client's 10k records changes mean recall by at most 0.8 points relative to fine-tuning across all clients (Table 2); only the failing Q arm moves more, in both directions. We did not test skewed or non-IID partitions, where one client may not cover all attack families.

Unseen hosts (E6). On the host-grouped TON_IoT split (Table 4) the compact INT8 models lose 1.6–1.9 points (R, S80, QC) to 4.4 points (P) of recall relative to the teacher, more than on the random split, while the over-budget F8 loses 0.06; the QAT collapse and its repair reproduce. The achieved test FAR departs from the 1% validation budget in both directions: the teacher reaches 4.77% in two runs and 0% in one, because 121 benign flows of one host cluster (4.77% of the 2,537 benign test flows) cross its threshold together, while every INT8 model with a bounded input range flags no benign test flow (0% test FAR); the failing Q and SQ reach 4.77% in two runs. At the 5% budget the achieved test FAR ranges from 0.3% to 5.8% across models and runs. With so few benign test hosts, unseen-host FAR is decided by single clusters and is a qualitative result.

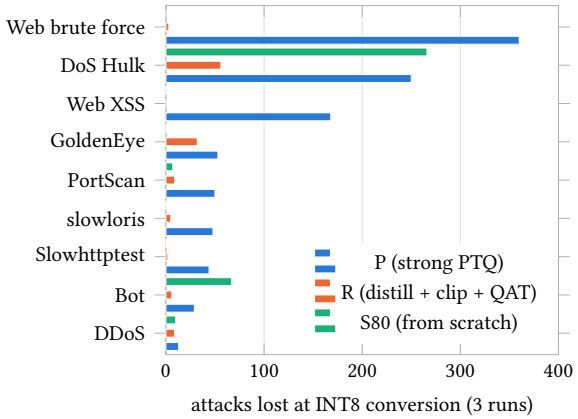


Figure 1. E2 on CIC-IDS2017: test attacks that the compact FP32 model detects and its INT8 version misses, at the 1% FAR operating point, by family (80 KB, sum over three runs; P and S80 as first selected, including per-channel exports). Totals: P 1,040, R 127, S80 369.

5 RQ2: Why Compression Fails

5.1 Where Decisions Change (E2)

Table 5 counts the test attacks whose decision changes between stages. Pruning and fine-tuning move thousands of decisions in both directions; the INT8 conversion moves few, except when the input range is wrong. For Q the loss happens in QAT itself, before conversion: 45,603 attacks (three runs) are lost between the compact FP32 and the QAT float model and 15 in the conversion. R's conversion loses 127 attacks where P's loses 1,040 (Figure 1): P's losses concentrate in the small web-attack families (brute force 360, XSS 168) and DoS Hulk, which R keeps; S80 loses mostly DoS Hulk and Bot flows. Two of P's three largest losses are among the smallest families (1,507 web brute-force and 652 XSS records in the whole dataset): aggregate recall hides where a coarse INT8 model gives up.

5.2 Input Range versus Hidden Range (E3)

To isolate the cause we fix the compact FP32 model and the calibration draw and vary only the ranges: input range min/max or clipped to ±5 SD, crossed with hidden ranges min/max or 99.99th percentile. The ranges are written into the quantizers of the unchanged FP32 model (no training), and the TFLite converter turns every cell into an exported integer model: float32 input → QUANTIZE → four FULLY_CONNECTED layers with int8 activations, per-tensor int8 weights and int32 biases → LOGISTIC (int8) → DEQUANTIZE, executed by the TFLite interpreter (TensorFlow 2.21) on the host. All E3 and E4 INT8 and 16-bit results are exported models of this kind; the only

Table 3. Principal deployable models (80 KB arms): attack recall (%) at a 1% validation FAR budget, mean ± SD over three runs, with the achieved test FAR (%) in brackets; size of the INT8 export; median invoke latency on the ESP32-S3 / ESP32 (TFLM 1.0.0, seed-1 export; TON_IoT/ CIC-IDS2017); federated training cost. The teacher F is the float reference. Exported files, SHA-256 and conversion settings per run: Supplement S1, S8.

model	KB (TON_IoT/CIC-IDS2017)	TON_IoT random	CIC-IDS2017 random	TON_IoT unseen hosts	S3 ms	ESP32 ms	training cost
F (FP32)	730/800	96.68±1.38 [0.84]	96.89±2.05 [0.98]	98.20±0.11 [3.18]	–	–	60 rounds (3.4 / 40.3 min)
P	56–57/65	96.57±1.89 [0.98]	99.12±0.37 [0.92]	93.81±0.27 [0.00]	5.55 / 6.47	13.18 / 15.36	F + 3 fine-tuning rounds
QC	57/65	97.86±0.44 [1.01]	99.57±0.10 [0.94]	96.30±0.87 [0.00]	5.55 / 6.48	13.20 / 15.27	F + 3 fine-tuning + 2 QAT rounds
R	57/65	98.51±0.11 [1.16]	99.60±0.15 [0.92]	96.57±0.62 [0.00]	5.56 / 6.48	13.19 / 15.38	F + 3 distilled fine-tuning + 2 QAT rounds
S80	56–57/65	97.57±0.52 [1.01]	99.57±0.24 [0.97]	96.48±0.94 [0.00]	5.56 / 6.47	13.35 / 15.37	60 rounds (2.7 / 25.9 min)
SQC	57/65	97.71±1.09 [1.01]	99.59±0.07 [0.97]	95.96±1.60 [0.00]	–	–	S80 + 2 QAT rounds

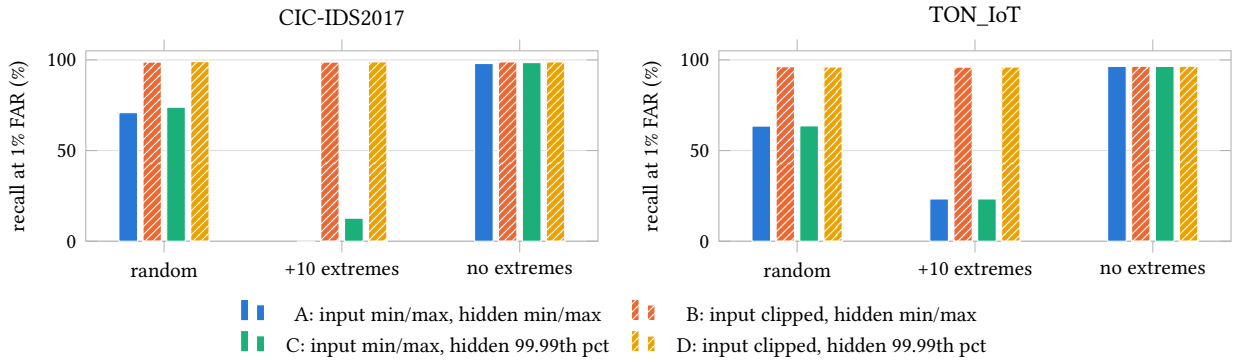


Figure 2. E3: recall of the pruned and fine-tuned 80 KB model when only the quantization ranges change (same FP32 model and calibration draw; mean over three runs and three draws). Hatched bars clip the input range to ±5 SD. Values in Table 6.

Table 4. TON_IoT, unseen source hosts (E6): recall (%) at a 1% validation FAR budget, 80 KB, three runs, with the mean achieved test FAR (%) in brackets (F, Q and SQ: 4.77 in two runs, 0 in one; every other model 0 in every run). F8, P, S80 with per-tensor strong PTQ. Per run and at 5%: Supplement S1.

F	F8	R	S80	QC
98.20±0.11 [3.18]	98.14±0.17 [0]	96.57±0.62 [0]	96.48±0.94 [0]	96.30±0.87 [0]
SQC	P	Q	SQ	
95.96±1.60 [0]	93.81±0.27 [0]	52.9±45.8 [3.18]	56.9±49.2 [3.18]	

Table 5. Test attacks lost / recovered between stages at the 1% FAR operating point (sum over three runs, 80 KB; P as first selected, including per-channel exports). Families with the most losses in brackets.

arm	stage	lost	rec.	main losses
CIC-IDS2017				
F8	teacher→INT8	573	7,125	DoS Hulk 384, Bot 73
P	teacher→pruned+FT	759	7,124	DoS Hulk 213, Web brute force 185
P	FP32→INT8	1,040	268	Web brute force 360, DoS Hulk 250
R	FP32→INT8	127	532	DoS Hulk 56, GoldenEye 32
Q	FP32→QAT float	45,603	21	PortScan 17,713, DDoS 13,079
Q	QAT float→INT8	15	1,051	
TON_IoT				
P	teacher→pruned+FT	859	645	dos 321, xss 197, mitm 103
P	FP32→INT8	336	415	dos 225, ddos 46
R	FP32→INT8	104	46	scanning 73

fake-quantized simulation in the paper is the "QAT float" stage of E2. Each cell is calibrated with three random 2,000-record draws, the same draws with the client's ten most extreme records swapped in, and draws without any record beyond 10 SD (Figure 2, Table 6).

The input range decides the outcome; the hidden range barely matters (A≈C, B≈D), and fewer than 0.05% of hidden values saturate in any cell. Ten extreme records among 2,000 widen the CIC-IDS2017 input step to 4.7 SD, so ordinary flows quantize to a handful of levels and every attack is missed. Clipping the input range makes the result independent of the draw. The teacher and the from-scratch models behave the same. QAT fails for the same reason: its input quantizer tracks the moving min/max of unclipped training batches, so it learns the wide range itself, and QAT on clipped inputs (QC, SQC) is as good as the best PTQ arms (Table 2). Clipping has a cost: ±5 SD saturates at least one feature in 28% of CIC-IDS2017 test rows (7.6% on TON_IoT), and for the CIC-IDS2017 teacher B and D are 1.5 points below extreme-free min/max calibration (A, last row), the gentler remedy.

Table 6. E3: recall (%) at 1% FAR of the pruned and fine-tuned 80 KB model, by input range (rows A/C: min/max; B/D: clipped ±5 SD) and hidden range (A/B: min/max; C/D: 99.99th percentile), mean over three runs and three draws. Step: input quantization step of A, in SD.

	calibration draw	A	B	C	D	step
CIC-IDS2017	random	70.9	98.9	73.9	99.1	0.15
	+ 10 extreme records	**0.0**	98.9	12.7	99.0	4.68
	none beyond 10 SD	98.0	99.0	98.5	99.0	0.05
TON_IoT	random	63.5	96.3	63.6	96.3	0.53
	+ 10 extreme records	23.3	96.1	23.3	96.2	0.74
	none beyond 10 SD	96.4	96.6	96.4	96.6	0.05

Table 7. E4: recall (%) at 1% FAR, three runs. Clip: FP32 with inputs clipped to ±5 SD. Grid: FP32 with its output rounded to the INT8 grid. INT8 and 16-bit: exported TFLite models run by the host interpreter; 16-bit (int16 activations, INT8 weights) is a host-only result.

	model	FP32	clip	grid	INT8	16-bit
TON_IoT	teacher F	96.68	98.40	96.65	98.28	98.40
	pruned+FT	96.24	97.35	96.01	96.59	97.34
	S80	96.98	97.52	96.90	97.36	97.52
CIC-IDS2017	teacher F	96.89	96.76	96.76	99.40	96.21
	pruned+FT	99.39	99.18	99.28	98.99	99.29
	S80	97.31	97.18	97.26	99.45	96.73

5.3 Output and Activation Resolution (E4)

Table 7 separates the remaining INT8 effects. Rounding only the FP32 output to the INT8 output grid (step 1/256) costs at most 0.25 points: the output grid is not the problem. The INT8 models, however, produce few distinct outputs on the validation slices (23–35 for the pruned CIC-IDS2017 models vs. 241–245 for the rounded FP32 output), so up to about 1,000 benign validation records share the output value just below the threshold. The 1% budget can then only be met by jumping over the whole tie group (validation FAR 0.74% instead of 1%), which costs recall. Exporting the logit instead of the sigmoid gives identical decisions: the bottleneck is the INT8 logit and the activations before it. With 16-bit activations (an exported int16x8 model: int16 activations, per-channel int8 weights, int64 biases, run by the host interpreter) the tie group shrinks to about 10 records and most of the loss returns (CIC-IDS2017: 462 lost attacks become 131). This recovery is a host result: no device runtime we tested executed the 16-bit model correctly (§7). On TON_IoT, where INT8 gains over FP32, clipping the FP32 inputs to ±5 SD gives the same gain: it is input saturation. On CIC-IDS2017 the INT8 gain of F and S80 (+2.1–2.5 points, the PortScan cluster of §4) disappears with 16-bit activations and is not reproduced by clipped FP32 inputs: INT8 rounding moves those flows across the teacher's threshold, an operating-point effect rather than a robust improvement.

6 RQ3: Remedies and Their Cost

Table 8 lists what worked, its effect and its cost. Bounding the input range is the remedy that matters: it is free on the device, needs no extra client data, and turns the two failure modes (min/max calibration with extreme records, QAT on unclipped inputs) into the best arms. Clipping is not free in accuracy: ±5 SD saturates at least one feature in 28% of CIC-IDS2017 and 7.6% of TON_IoT test records, and costs the CIC-IDS2017 teacher 1.5 points against extreme-free min/max calibration, which is the gentler variant for models that use the tails. The complete recovery recipe R (distilled fine-tuning on clipped inputs, then QAT on clipped inputs) improves on P by 1.9 points on TON_IoT and 0.5 on CIC-IDS2017. We did not isolate its components: QC, which has the clipped QAT but no distillation, is within 0.65 points of R on TON_IoT and 0.03 on CIC-IDS2017, so we attribute the gain to the recipe as a whole. Training a compact model from scratch removes the compression step altogether at the price of a second federated training (Table 3). 16-bit activations remove most of the remaining tie loss on the host, but no tested device runtime executed the 16-bit model correctly, so their deployment cost is not established. In our label-balanced partition, one client's 10k records sufficed for fine-tuning.

7 On-Device Agreement and Cost (E7)

A host result is only useful if the device reproduces it. We stream test inputs over serial to firmware (PlatformIO, TensorFlowLite_ESP32 1.0.0, TFLM reference kernels; the current runtime below) that runs each embedded model and returns the exact float32 output bits and per-stage timings, and compare with the host TFLite interpreter. Per model, the inputs are 1,500 random test records, the test records within two output steps of a validation threshold of any model, 800 records with a feature beyond 5 SD, 400 records with one feature set to ±50 or ±1,000 SD, and 10 zero vectors (4,462 on TON_IoT, 5,902 on CIC-IDS2017).

Table 9 shows two classes of models. Per-tensor INT8 models reproduce the host: at least 96% of outputs are within one output step, random, out-of-range and synthetic-extreme inputs get the same decision with at most two exceptions in 2,700 per model and budget, and the other disagreements are inputs we chose to lie within two steps of the threshold. Across all per-tensor models the device adds one missed attack in total (one CIC-IDS2017 record, S80 at the 1% budget); every other disagreement adds or avoids a false alarm, adds a detection, or concerns an unlabelled synthetic probe (classification per model, budget and input category in Supplement S3; stream completeness and resets in S5). Models

Table 8. Remedies, effect on recall at a 1% validation FAR budget (80 KB), and cost.

remedy	effect	cost
clip calibration inputs ±5 SD, or drop records beyond 10 SD	CIC-IDS2017 0.0→98.9–99.0 with extreme records in the draw	none on device; clipping saturates 8–28% of test records, −1.5 for the CIC-IDS2017 teacher
QAT on clipped inputs (QC, SQC)	TON_IoT 55.4→97.9, CIC-IDS2017 81.9→99.6	2 QAT rounds; as above
complete recovery recipe (R vs. P)	TON_IoT +1.9, CIC-IDS2017 +0.5; components not isolated	teacher, 3 distilled fine-tuning + 2 QAT rounds
compact model from scratch (S80)	equal to R on CIC-IDS2017 and unseen hosts, −0.9 on TON_IoT	second federated training
16-bit activations	CIC-IDS2017 lost attacks 462→131 (host only)	not established: no tested runtime executed the model correctly
per-tensor weight scales	device agreement on TFLM 1.0.0 (§7)	none

Table 9. E7: share of outputs within one INT8 output step of the host, decisions agreeing at the 1% FAR threshold on all inputs and on the 1,500 random records, and median invoke latency on an ESP32-S3 and an ESP32 (both 240 MHz). The two chips return bit-identical outputs. Arena 1.4 KB (1.8 KB for 16-bit).

model	weights	≤1 step	all inputs	random	S3 ms	ESP32 ms
TON_IoT R80	per-tensor	97.9%	4,462/4,462	1,500/1,500	5.56	13.19
TON_IoT QC80	per-tensor	97.4%	4,456/4,462	1,500/1,500	5.55	13.20
TON_IoT S80, per-tensor export	per-tensor	99.2%	4,462/4,462	1,500/1,500	5.56	13.35
TON_IoT P80, per-tensor export	per-tensor	99.8%	4,461/4,462	1,500/1,500	5.55	13.18
TON_IoT S80 (strong PTQ)	per-channel	4.9%	2,209/4,462	443/1,500	5.56	13.21
TON_IoT P80 (strong PTQ)	per-channel	40.9%	3,141/4,462	705/1,500	5.56	13.29
CIC-IDS2017 R80	per-tensor	96.4%	5,882/5,902	1,500/1,500	6.48	15.38
CIC-IDS2017 S80	per-tensor	97.2%	5,898/5,902	1,500/1,500	6.47	15.37
CIC-IDS2017 P80	per-tensor	97.9%	5,878/5,902	1,499/1,500	6.47	15.36
CIC-IDS2017 16-bit act.	per-channel	67.5%	4,952/5,902	1,425/1,500	7.39	16.11

whose Dense weights carry per-channel scales, the TFLite converter's default and what strong PTQ selected for two TON_IoT models, run without any error on this runtime but disagree with the host on half of the random records, mostly as missed attacks; re-exporting the same models with per-tensor scales restores agreement (4,462 / 4,462), and the same per-channel file agrees under the current runtime (below). The 16-bit-activation models are not reproduced either.

Current runtime. For a targeted comparison we ran three of the files, with the same 4,462 TON_IoT inputs, on the classic ESP32 through the current Espressif port, esp-tflite-micro 1.4.2 on ESP-IDF 5.5, once with the esp-nn kernels and once with reference kernels (Table 10; converter TensorFlow 2.21, all files declare QUANTIZE v1, FULLY_CONNECTED v4, LOGISTIC v2, DEQUANTIZE v2). On this board and these inputs, the per-channel S80 export, which agreed on 2,209 decisions under TFLM 1.0.0, agreed on 4,459 under the current runtime, with three additional detections and no additional misses. The per-tensor S80 export kept complete decision agreement and gave bit-identical outputs under both runtimes. The 16-bit model returned one constant output for every input, equal to the sigmoid of its output-layer bias; we did not diagnose the cause. The two kernel settings gave bit-identical outputs for all three files. We did not run the other models of Table 9 or the ESP32-S3 on the current runtime, so its agreement for those configurations is not established. Neither the export format nor a passing host evaluation tells which runtime will execute a model correctly, which is why we recommend this streaming check, on the exact exported file and runtime, before reporting on-device accuracy.

Table 10. The same files and 4,462 TON_IoT inputs on the same ESP32 through TFLM 1.0.0 (old) and esp-tflite-micro 1.4.2 / ESP-IDF 5.5 (current; esp-nn and reference kernels give bit-identical outputs): outputs equal to the host, decisions agreeing at the 1% FAR threshold, extra misses, median invoke latency.

model	runtime	outputs = host	decisions	extra misses	ms
S80, per-channel	old	170	2,209	2,069	13.27
S80, per-channel	current	4,377	4,459	0	10.43
S80, per-tensor	old	4,099	4,462	0	13.39
S80, per-tensor	current	4,099	4,462	0	10.30
16-bit act., per-channel	old	425	3,379	933	13.87
16-bit act., per-channel	current	0 (constant)	1,894	2,278	12.10

The ESP32 (Xtensa LX6) returns bit-identical outputs to the ESP32-S3 (LX7) for all 14 models and 72,548 inputs, so every deviation from the host comes from the runtime, not the chip; it ran the whole stream without a reset, while the S3 restarted a few times at random inputs and resumed. A flow takes 5.6 ms (TON_IoT, 43 features) and 6.5 ms (CIC-IDS2017, 78 features) on the ESP32-S3 and 13.2 ms and 15.4 ms on the ESP32; input quantization takes 3–13 μs and the arena 1.4 KB. The current runtime ran the two S80 files 22% faster on the ESP32 (10.3–10.4 ms) with 1.9 KB (per-tensor) and 5.5 KB (per-channel) of arena; in both builds the free heap changes by at most 32 bytes across model selections (Supplement S4).

8 Findings and Recommendations

Table 11 links each principal finding to its evidence and to what a practitioner should do.

Table 11. Principal findings, the evidence in this paper and its supplement, and practical implications.

finding	evidence	implication
A compact model trained federatively at the target size matches the complete recovery recipe on CIC-IDS2017 and on unseen hosts; the recipe is 0.5–1.6 points better on the random TON_IoT split, at a slightly higher test FAR. Same size and latency.	Tables 2, 4, 3; run-paired differences (S2); training times (S8).	If the memory budget is known before training, train the compact model and report it as the baseline a compression pipeline must beat; use the recipe when a teacher already exists and its gain on the target data is confirmed.
The input quantization range causes the collapses of PTQ and QAT; bounding it prevents them.	E3 2×2 with controlled calibration draws (Table 6, Figure 2); QC/SQC vs. Q/SQ (Table 2).	Clip calibration and QAT inputs (±5 SD) or drop records beyond 10 SD from the calibration draw; check the exported input step in SD (0.15 SD or more always collapsed recall at 1% FAR here, 0.05 SD never did); weigh the saturation clipping causes (8–28% of records).
Detection must be verified for the exact exported file and runtime.	E7 on two chips (Table 9, S3, S5); runtime comparison on the classic ESP32 (Table 10, S6).	Stream near-threshold and out-of-range inputs through the device and compare with the host on the same file; export per-tensor weights for TFLM 1.0.0; treat 16-bit activations as unverified on device.
A validation FAR budget does not fix the achieved test FAR.	0.66–1.27% at the 1% budget on random splits, 0–4.77% on unseen hosts (S1); float32 threshold pitfall (§3.4).	Choose thresholds on real client-held validation records in the output's precision, and report the achieved test FAR next to the budget.
Losses concentrate in small attack families.	E2 (Table 5, Figure 1).	Report losses per attack family; aggregate recall hides them.

9 Threats to Validity

Scope: two datasets, binary detection, one model family (MLP) and four label-balanced clients; the one-client result holds for this partition only. *Splits*: the revised test records were in the training pool of our earlier, exploratory pipeline, whose models guided our hypotheses; they were excluded from training, threshold selection and calibration of every reported model, and several analyses were added after test results (§3.1). CIC-IDS2017 has no host field, so the unseen-host test is TON_IoT-only, and its benign test set comes from few hosts, making its test FAR depend on single clusters. *Seeds*: three federated runs per configuration; the float teacher's 1%-FAR recall varies by up to 2 points across runs, and claims about it are stated as sensitivity. *Attribution*: we did not isolate the components of the recovery recipe R. *Labels*: CIC-IDS2017 contains labelling errors [11]; we drop impossible values and conflicting labels but do not relabel. *Runtime*: TFLM 1.0.0 on two ESP32 chips for all device models; the current esp-tflite-micro 1.4.2 only for three files on the classic ESP32, so its behaviour on the other models and on the ESP32-S3 (where esp-nn's optimized kernels apply) is not established, and the cause of the 16-bit failure is not diagnosed. *Threat model*: no adversarial or poisoning study [36].

10 Conclusion

A federated intrusion detector can be compressed to 57–65 KB INT8 and keep its detection at a 1% validation false-alarm budget on two datasets and three runs, using only client-held data, at achieved test false-alarm rates of 0.9–1.2%; on unseen hosts the compact models lose 1.6–4.4 points of recall relative to the float teacher. A compact model trained federatively from scratch is a sufficient choice on CIC-IDS2017 and on unseen hosts, while the complete recovery recipe gives a consistent gain on the random TON_IoT split. When compression fails, the input quantization range is the cause, for post-training quantization and QAT alike, and bounding it prevents the failure at no device cost. The remaining loss sits in coarse activations at the operating point; 16-bit activations recover it on the host only. Finally, the exact exported file and runtime decide what the device does: under TFLM 1.0.0 per-tensor INT8 models agree with the host on both chips and per-channel exports do not, the current Espressif runtime restored agreement for the per-channel model we tested on the classic ESP32, and the 16-bit model ran correctly on neither.

References

[1] Luigi Atzori, Antonio Iera, and Giacomo Morabito. 2010. The Internet of Things: A Survey. *Computer Networks* 54, 15 (2010), 2787–2805. https://doi.org/10.1016/j.comnet.2010.05.010

[2] Colby R. Banbury, Vijay Janapa Reddi, Max Lam, William Fu, Amin Fazel, Jeremy Holleman, Xinyuan Huang, Robert Hurtado, David Kanter, Anton Lokhmotov, David Patterson, Danilo Pau, Jae sun Seo, Jeff

Sieracki, Urmish Thakker, Marian Verhelst, and Poonam Yadav. 2020. Benchmarking TinyML Systems: Challenges and Direction. *arXiv preprint arXiv:2003.04821* (2020).

[3] Ron Banner, Yury Nahshan, and Daniel Soudry. 2019. Post Training 4-bit Quantization of Convolutional Networks for Rapid-Deployment. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[4] Othmane Belarbi, Theodoros Spyridopoulos, Eirini Anthi, Ioannis Mavromatis, Pietro Carnelli, and Aftab Khan. 2023. Federated Deep Learning for Intrusion Detection in IoT Networks. In *IEEE Global Communications Conference (GLOBECOM)*. 237–242. https://doi.org/10.1109/GLOBECOM54140.2023.10437860

[5] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchi Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, and Nicholas D. Lane. 2020. Flower: A Friendly Federated Learning Research Framework. *arXiv preprint arXiv:2007.14390* (2020).

[6] Zhenpeng Chen, Huihan Yao, Yiling Lou, Yanbin Cao, Yuanqiang Liu, Haoyu Wang, and Xuanzhe Liu. 2021. An Empirical Study on Deployment Faults of Deep Learning Based Mobile Applications. In *IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*.

[7] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. 2018. PACT: Parameterized Clipping Activation for Quantized Neural Networks. *arXiv preprint arXiv:1805.06085* (2018).

[8] Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, Ian Nappier, Meghna Natraj, Tiezhen Wang, Pete Warden, and Rocky Rhodes. 2021. TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 3. 800–811.

[9] Enmao Diao, Jie Ding, and Vahid Tarokh. 2021. HeteroFL: Computation and Communication Efficient Federated Learning for Heterogeneous Clients. In *International Conference on Learning Representations (ICLR)*.

[10] Yutao Dong, Qing Li, Kaidong Wu, et al. 2023. HorusEye: A Real-time IoT Malicious Traffic Detection Framework using Programmable Switches. In *32nd USENIX Security Symposium*.

[11] Gints Engelen, Vera Rimmer, and Wouter Joosen. 2021. Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study. In *IEEE Security and Privacy Workshops (SPW)*. 7–12. https://doi.org/10.1109/SPW53761.2021.00009

[12] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. 2020. Learned Step Size Quantization. In *International Conference on Learning Representations (ICLR)*.

[13] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2021. A Survey of Quantization Methods for Efficient Neural Network Inference. *arXiv preprint arXiv:2103.13630* (2021).

[14] Sridhar Gopinath, Nikhil Ghanathe, Vivek Seshadri, and Rahul Sharma. 2019. Compiling KB-Sized Machine Learning Models to Tiny IoT Devices. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.

[15] Kartik Gupta, Marios Fournarakis, Matthias Reisser, Christos Louizos, and Markus Nagel. 2023. Quantization Robust Federated Learning for Efficient Inference on Heterogeneous Devices. *Transactions on Machine Learning Research (TMLR)* (2023). arXiv:2206.10844.

[16] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In *International Conference on Learning Representations (ICLR)*.

[17] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531* (2015).

[18] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. 2019. Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification. *arXiv preprint arXiv:1909.06335* (2019).

[19] Hong Huang, Lan Zhang, Chaoyue Sun, Ruogu Fang, Xiaoyong Yuan, and Dapeng Wu. 2023. Distributed Pruning Towards Tiny Neural Networks in Federated Learning. In *IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*. 190–201.

[20] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2704–2713.

[21] Shikhar Jaiswal, Rahul Kranti Kiran Goli, Aayan Kumar, Vivek Seshadri, and Rahul Sharma. 2023. MinUn: Accurate ML Inference on Microcontrollers. In *Proceedings of the 24th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES)*. 26–39.

[22] Purvish Jajal, Wenxin Jiang, Arav Tewari, Erik Kocinare, Joseph Woo, Anusha Sarraf, Yung-Hsiang Lu, George K. Thiruvathukal, and James C. Davis. 2024. Interoperability in Deep Learning: A User Survey and Failure Analysis of ONNX Model Converters. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. https://doi.org/10.1145/3650212.3680374

[23] Jakub Konečný, H. Brendan McMahan, Felix X. Yu, Peter Richtárik, Ananda Theertha Suresh, and Dave Bacon. 2016. Federated Learning: Strategies for Improving Communication Efficiency. *arXiv preprint arXiv:1610.05492* (2016).

[24] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova, and Benjamin Turnbull. 2019. Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset. *Future Generation Computer Systems* 100 (2019), 779–796. https://doi.org/10.1016/j.future.2019.05.041

[25] Mingyan Li et al. 2025. Cognitive IoT and Edge Computing for Intrusion Detection with Federated TinyML. In *IEEE World AI IoT Congress (AIIoT)*. 677–684.

[26] Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. 2020. MCUNet: Tiny Deep Learning on IoT Devices. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[27] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Chuang Gan, and Song Han. 2022. On-Device Training Under 256KB Memory. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[28] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal Loss for Dense Object Detection. In *IEEE International Conference on Computer Vision (ICCV)*. 2980–2988.

[29] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS) (PMLR, Vol. 54)*. 1273–1282.

[30] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. In *Network and Distributed System Security Symposium (NDSS)*.

[31] Nour Moustafa. 2021. A New Distributed Architecture for Evaluating AI-Based Security Systems at the Edge: Network TON_IoT Datasets. *Sustainable Cities and Society* 72 (2021), 102994. https://doi.org/10.1016/j.scs.2021.102994

[32] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. 2021. A White Paper on Neural Network Quantization. *arXiv preprint arXiv:2106.08295* (2021).

[33] Markus Nagel, Marios Fournarakis, Yelysei Bondarenko, and Tijmen Blankevoort. 2022. Overcoming Oscillations in Quantization-Aware Training. In *Proceedings of the 39th International Conference on Machine Learning (ICML) (PMLR, Vol. 162)*. 16318–16330.

[34] Markus Nagel, Mart van Baalen, Tijmen Blankevoort, and Max Welling. 2019. Data-Free Quantization Through Weight Equalization and Bias Correction. In *IEEE/CVF International Conference on Computer Vision (ICCV)*. 1325–1334.

[35] Thien Duc Nguyen, Samuel Marchal, Markus Miettinen, Hossein Fereidooni, N. Asokan, and Ahmad-Reza Sadeghi. 2019. DÏoT: A Federated Self-learning Anomaly Detection System for IoT. In *IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. 756–767.

[36] Thien Duc Nguyen, Phillip Rieger, Huili Chen, Hossein Yalame, Helen Möllering, Hossein Fereidooni, Samuel Marchal, Markus Miettinen, Azalia Mirhoseini, Shaza Zeitouni, Farinaz Koushanfar, Ahmad-Reza Sadeghi, and Thomas Schneider. 2022. FLAME: Taming Backdoors in Federated Learning. In *31st USENIX Security Symposium*.

[37] Antonio Polino, Razvan Pascanu, and Dan Alistarh. 2018. Model Compression via Distillation and Quantization. In *International Conference on Learning Representations (ICLR)*.

[38] Amirhossein Reisizadeh, Aryan Mokhtari, Hamed Hassani, Ali Jadbabaie, and Ramtin Pedarsani. 2020. FedPAQ: A Communication-Efficient Federated Learning Method with Periodic Averaging and Quantization. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS) (PMLR, Vol. 108)*. 2021–2031.

[39] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 108–116. https://doi.org/10.5220/0006639801080116

[40] TensorFlow Model Optimization. 2021. Collaborative Optimization: Pruning-Preserving Quantization Aware Training. https://www.tensorflow.org/model_optimization/guide/combine/collaborative_optimization.

[41] Hao Wu, Patrick Judd, Xiaojie Zhang, Mikhail Isaev, and Paulius Micikevicius. 2020. Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation. *arXiv preprint arXiv:2004.09602* (2020).

[42] Guangmeng Zhou, Zhuotao Liu, Chuanpu Fu, Qi Li, and Ke Xu. 2023. An Efficient Design of Intelligent Network Data Plane. In *32nd USENIX Security Symposium*.

A Audit of the Earlier Pipeline

Checks (docs/AUDIT.md): **D1** record ids and split indices recoverable (fixed: per-run manifests with ids and hashes); **D2** threshold-validation records excluded from training (failed: the slice was part of client 0's training data); **D3** validation uses real records (failed: 37.8% / 35.1% synthetic); **D4** no record or synthetic descendant crosses a partition (failed: SMOTE before partitioning, ~255k / 11.4k synthetic rows per client); **D5** scaler fit on training data only (passed); **D6** test split untouched by development (failed; a fresh random split still shared 20% of its records with the old one, so the new test split excludes them); **D7** duplicates and conflicting labels (fixed); **C3** SMOTE on label-encoded fields (failed: 4.7% of synthetic TON_IoT rows impossible); **N1** extreme records (the most extreme CIC-IDS2017 records were overflow errors, now dropped); **N2** input conversion (passed); **N3** BatchNorm folding (passed); **N4** no test data in training (passed); **N5** seeds recorded (fixed). Results of the earlier pipeline were exploratory and are not reported here.

B Exact BatchNorm Folding

Each hidden layer is Dense→ReLU→BatchNorm→Dropout. BatchNorm at inference is $s \odot h + t$ with $s = \gamma/\sqrt{\sigma^2 + \varepsilon}$ and $t = \beta - s \odot \mu$. Since it follows the ReLU it is folded into the next Dense layer (W, b): $W' = \mathrm{diag}(s)W$, $b' = b + t^\top W$ (deviation $\leq 2 \cdot 10^{-5}$). Folding it into the preceding layer, a common shortcut, changed 11.6% of decisions on our layout.

C Toolchain Pitfalls

Six failures produced plausible numbers without any error: int8 weight exchange without scales in FL; QAT-wrapper stripping that copied no weights; BatchNorm folded on the wrong side; pruning that reset BatchNorm statistics; float64 thresholds compared with float32 INT8 outputs (§3.4); and per-channel exports executed incorrectly, without any error, by an older microcontroller runtime (§7). The checks that catch them: dequantize-requantize equality at the receiver, copy-by-name with output comparison, equivalence assertions after folding, validation FAR asserted against the budget, and device streaming against the host.